INFO8006 Introduction to Artificial Intelligence

Exercise session 6: Machine learning

Recap

Problem statement

Let $\mathbf{d} \sim p(\mathbf{x}, y)$ be a dataset of N example input-output pairs

$$\mathbf{d} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_N, y_N)\},$$

where $\mathbf{x}_i \in \mathbb{R}^d$ are d-dimensional vectors representing the input values and $y_i \in \mathcal{Y}$ are the corresponding output values. From this data, we want to identify a probabilistic model $p_\theta(y \mid \mathbf{x})$ that best explains the data. The problem is a *regression* problem if $y \in \mathbb{R}$ and a *classification* problem if $y \in \{0, 1, \ldots, C-1\}$.

Linear regression

Linear regression models $p(y \mid \mathbf{x}) = \mathcal{N}(y \mid \mathbf{w}^T\mathbf{x} + b, \sigma^2)$. Its maximum likelihood estimate minimizes the sum of squared errors $\sum_{i=1}^N (y_i - (\mathbf{w}^T\mathbf{x}_i + b))^2$. Appending a constant feature 1 to $\mathbf{x}$ to absorb b into $\mathbf{w}$, it is $\mathbf{w}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$, where $\mathbf{X}$ stacks the inputs $\mathbf{x}_i^T$ and $\mathbf{y}$ the outputs y_i.

Logistic regression

Logistic regression models $P(Y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x} + b)$, with the sigmoid $\sigma(z) = 1/(1+\exp(-z))$. Its maximum likelihood estimate minimizes the *cross-entropy* $\sum_{i=1}^N -y_i \log \sigma(\mathbf{w}^T\mathbf{x}_i + b) - (1 - y_i)\log(1 - \sigma(\mathbf{w}^T\mathbf{x}_i + b))$, which has no closed-form minimizer.

Gradient descent

To minimize a loss function $\mathcal{L}(\theta)$, *gradient descent* updates the model parameters iteratively using the update rule

$$\theta_{t+1} = \theta_t - \alpha \nabla_\theta \mathcal{L}(\theta_t),$$

where θ_0 are the initial parameters of the model and α is the *learning rate*. When $\mathcal{L}(\theta) = \frac{1}{N}\sum_i \ell(y_i, f(\mathbf{x}_i; \theta))$, *stochastic gradient descent* replaces the gradient by its average over a random minibatch of the pairs.

Multilayer perceptron

A *shallow network* with input $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$, q hidden units and one output computes

$$h_j = \sigma\Big(\sum_{i=0}^{d_{\text{in}}-1} w_{ji}x_i + b_j\Big) \quad \text{for } j = 0, \ldots, q-1, \qquad y = \sum_{j=0}^{q-1} v_j h_j + c,$$

where σ is an activation function. In matrix form, a hidden layer computes

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}),$$

where $\mathbf{W} \in \mathbb{R}^{q \times d_{\text{in}}}$ is the *weight matrix*, of entries w_{ji}, and $\mathbf{b} \in \mathbb{R}^q$ is the *bias vector*. Layers composed in series form a deep network with K hidden layers,

$$\begin{aligned} \mathbf{h}_0 &= \mathbf{x} \\ \mathbf{h}_k &= \sigma(\mathbf{W}_k \mathbf{h}_{k-1} + \mathbf{b}_k) \quad \text{for } k = 1, \ldots, K \\ \mathbf{y} &= \sigma_{\text{out}}(\mathbf{W}_y \mathbf{h}_K + \mathbf{b}_y), \end{aligned}$$

where $\mathbf{W}_k \in \mathbb{R}^{q_k \times q_{k-1}}$ is the weight matrix and $\mathbf{b}_k \in \mathbb{R}^{q_k}$ is the bias vector of the k-th layer. The resulting function $f(\mathbf{x}; \theta)$, where θ denotes the set of all model parameters, is a *multilayer perceptron*. With ReLU units, it is piecewise linear: a shallow network with one input has at most $q + 1$ linear regions, a deep network with K layers of q units up to $(q + 1)^K$.

Universal approximation theorem

Let $\sigma : \mathbb{R} \to \mathbb{R}$ be continuous and not a polynomial. For every compact set $K \subset \mathbb{R}^{d_{\text{in}}}$, every continuous function $f : K \to \mathbb{R}$ and every $\epsilon > 0$, there exist q, w_{ji}, b_j, v_j and c such that

$$\sup_{\mathbf{x} \in K} \left| f(\mathbf{x}) - \Big(\sum_{j=0}^{q-1} v_j \, \sigma\Big(\sum_{i=0}^{d_{\text{in}}-1} w_{ji} x_i + b_j \Big) + c \Big) \right| < \epsilon :$$

a shallow network with enough hidden units approximates any continuous function on a closed and bounded domain arbitrarily well.

Activation functions

Activation functions $\sigma : \mathbb{R} \to \mathbb{R}$ introduce non-linearities in the network. Common activation functions include

- the sigmoid $\sigma(x) = \frac{1}{1+\exp(-x)}$, which maps inputs to the range $(0, 1)$;
- the hyperbolic tangent $\sigma(x) = \tanh(x) = \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)}$, which maps inputs to the range $(-1, 1)$;
- the rectified linear unit $\sigma(x) = \text{ReLU}(x) = \max(0, x)$.

Output layers

The output layer $\mathbf{y} = \sigma_{\text{out}}(\mathbf{W}_y \mathbf{h}_K + \mathbf{b}_y)$ depends on the task:

- for regression, its width is the dimensionality d_{out} of the output and σ_{out} is the identity;
- for binary classification, its width is 1 and σ_{out} is the sigmoid, whose output is $P(Y = 1 \mid \mathbf{x})$;
- for classification into C classes, its width is C and σ_{out} is the *softmax*

$$\text{Softmax}(\mathbf{z})_i = \frac{\exp(z_i)}{\sum_{i'=0}^{C-1} \exp(z_{i'})} \quad \text{for } i = 0, \ldots, C-1,$$

  so that $\mathbf{y}$ is a vector of C probabilities $P(Y = i \mid \mathbf{x})$ summing to 1.

Loss functions

The parameters θ of a deep network $f(\mathbf{x}; \theta)$ are learned by minimizing a loss function $\mathcal{L}(\theta)$ over the dataset $\mathbf{d} = \{(\mathbf{x}_i, \mathbf{y}_i)\}$ of N input-output pairs. The loss function is derived from the likelihood:

- for regression, assuming a Gaussian likelihood, the loss is the *mean squared error*

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \|\mathbf{y}_i - f(\mathbf{x}_i; \theta)\|^2;$$

- for classification, assuming a categorical likelihood, the loss is the *cross-entropy*, the negative log-probability of the observed class,

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log f_{y_i}(\mathbf{x}_i; \theta).$$

Chain rule

The gradient of the loss is obtained by the chain rule. For example, for $h_1 = \sigma(w_1 x + b_1)$, $\hat{y} = w_2 h_1 + b_2$ and $\ell = (y - \hat{y})^2$, the chain $w_1 \to h_1 \to \hat{y} \to \ell$ gives

$$\frac{\partial \ell}{\partial w_1} = \frac{\partial \ell}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_1} \frac{\partial h_1}{\partial w_1} = -2(y - \hat{y})\, w_2\, \sigma'(w_1 x + b_1)\, x.$$

On the *computation graph* of the network (variables and the operations that produce them), the *forward pass* computes every intermediate value from the input to the loss; the *backward pass* then multiplies the local derivatives from the loss back to each parameter, reusing the values of the forward pass. *Automatic differentiation* implements this procedure for any program.

Generalization

We want a small loss on new pairs drawn from $p(\mathbf{x}, y)$, not on the pairs of $\mathbf{d}$. An honest estimate comes from a *test set* $\mathbf{d}_{\text{test}}$, held out from training. A model too small for the data *underfits* (both losses stay high); a model large enough to memorize $\mathbf{d}$ *overfits* (the training loss decreases while the test loss increases). *Hyperparameters* (width, depth, learning rate, minibatch size, number of epochs) are chosen on a *validation set*, also held out from training; the test set is used once, at the end.

Convolution

For the 1d input $\mathbf{x} \in \mathbb{R}^W$ and the convolutional kernel $\omega \in \mathbb{R}^w$, the discrete *convolution* $\mathbf{x} \circledast \omega$ is a vector of size $W - w + 1$ such that

$$(\mathbf{x} \circledast \omega)[i] = \sum_{m=0}^{w-1} \mathbf{x}_{m+i} \omega_m.$$

Technically, $\circledast$ is the cross-correlation: the kernel is not flipped. Padding the input with p zeros on each side results in an output of size $W - w + 1 + 2p$. For the 3d input tensor $\mathbf{x} \in \mathbb{R}^{C \times H \times W}$ with C channels and the 3d convolutional kernel $\omega \in \mathbb{R}^{C \times h \times w}$, the 2d convolution $\mathbf{x} \circledast \omega$ is a 2d tensor of size $(H - h + 1) \times (W - w + 1)$ such that

$$(\mathbf{x} \circledast \omega)[j, i] = \sum_{c=0}^{C-1} \sum_{n=0}^{h-1} \sum_{m=0}^{w-1} \mathbf{x}_{c,n+j,m+i} \omega_{c,n,m}.$$

Convolutional neural networks

A *convolutional layer* is defined by a set of K kernels ω_k of size $C \times h \times w$. It applies the 2d convolution to the input tensor $\mathbf{x}$ of size $C \times H \times W$ to produce a set of K feature maps $\mathbf{o}_k$, the K channels of its output. With a pooling area of size $h \times w$, *max-pooling* maps a tensor $\mathbf{x} \in \mathbb{R}^{C \times (rh) \times (sw)}$ to a tensor $\mathbf{o} \in \mathbb{R}^{C \times r \times s}$ such that

$$\mathbf{o}_{c,j,i} = \max_{n<h, m<w} \mathbf{x}_{c,hj+n,wi+m}.$$

The *receptive field* of a value is the patch of the input it depends on: a $w \times w$ kernel sees $w \times w$ values, and each further convolutional or pooling layer widens it. A *convolutional neural network* (CNN) uses convolutional layers, pooling layers and fully connected layers.

Exercises

In session: exercises 1, 4, 7.

Exercise 1 Travel-time heuristic (Exam, January 2022)
A delivery robot works in a building laid out as a 5×5 grid of rooms, where each cell (r, c) is a room with doors leading to the adjacent rooms. The robot must reach its charging station as fast as possible, but the station's position is unknown to it. Furthermore, some regions of the building are cluttered, and crossing rooms in these regions takes longer. Fortunately, the other robots of the fleet have logged their best times to the station, starting from a few different rooms. Due to rounding errors, the best times in the log are assumed to be measurements affected by additive Gaussian noise $\mathcal{N}(0, 1)$.

i	Starting room	Measured best time
1	$(4, 5)$	2.0
2	$(5, 3)$	3.5
3	$(3, 3)$	4.5
4	$(4, 1)$	7.0
5	$(1, 2)$	8.5

From the log, the robot's program learns a heuristic approximating the best time to get to the station, starting from room (r, c). It uses a small neural network as approximator, described by the following parametric function,

$$h(r, c; \phi) = \mathrm{ReLU}(rw_1 + cw_2 + w_3) + \mathrm{ReLU}(rw_4 + cw_5 + w_6)$$
$$\mathrm{ReLU}(z) = \max(z, 0),$$

where $\phi = (w_1, w_2, w_3, w_4, w_5, w_6)$ is the set of parameters/weights of the neural network.

(a) Among the following sets of parameters (A, B or C), which one would you use? Justify your answer.

Set	w_1	w_2	w_3	w_4	w_5	w_6
ϕ_A	-1.5	1	4	1	-1.5	6
ϕ_B	-1	1.5	3	0	-1	4
ϕ_C	-2	0.5	4.5	1.5	0	5

Solution. A set of parameters is better than another if it explains the data better, that is if the likelihood is higher. Our task is to find the set of parameters that maximizes the likelihood under the assumed probability model. In our case, the data $\mathbf{d}$ is the log and consists of independent position-time tuples (r_i, c_i, t_i). From the statement, we gather that the (best) time t is a function of the starting position (r, c), approximated by a neural network $h(r, c; \phi)$. We also learn that our time measurements are affected by Gaussian

noise.

$$\begin{aligned}
\phi_{\text{MLE}} &= \arg\max_{\phi} p(\mathbf{d} \mid \phi) \\
&= \arg\max_{\phi} \prod_{i=1}^{5} p(r_i, c_i, t_i \mid \phi) \\
&= \arg\max_{\phi} \prod_{i=1}^{5} p(t_i \mid r_i, c_i, \phi)\, p(r_i, c_i) \\
&= \arg\max_{\phi} \prod_{i=1}^{5} \mathcal{N}(t_i; h(r_i, c_i; \phi), 1) \\
&= \arg\max_{\phi} \sum_{i=1}^{5} \log\left[\frac{1}{\sqrt{2\pi}} \exp\left(-\frac{(t_i - h(r_i, c_i; \phi))^2}{2}\right)\right] \\
&= \arg\min_{\phi} \sum_{i=1}^{5} (t_i - h(r_i, c_i; \phi))^2
\end{aligned}$$

Now that we have an objective $L(\phi)$ to minimize, we can evaluate it for each of the proposed sets and select the best one.

$$\begin{aligned}
L(\phi_A) &= \sum_{i=1}^{5} (t_i - h(r_i, c_i; \phi_A))^2 = (2.0 - 5.5)^2 + (3.5 - 6.5)^2 + \cdots = 29.75 \\
L(\phi_B) &= 35.75 \\
L(\phi_C) &= 205.25
\end{aligned}$$

The best set is ϕ_A.

(b) You now assume a Gaussian prior $\mathcal{N}(0, 1)$ on each parameter. Which set of parameters in the table above would you now choose? Justify your answer.

Solution. Now that we hold a prior belief on the parameters, our task is to find the most likely set of parameters given the data, that is maximum a posteriori (MAP) estimation.

$$\begin{aligned}
\phi_{\text{MAP}} &= \arg\max_{\phi} p(\phi \mid \mathbf{d}) \\
&= \arg\max_{\phi} p(\mathbf{d} \mid \phi) p(\phi) \\
&= \arg\max_{\phi} \prod_{i=1}^{5} p(r_i, c_i, t_i \mid \phi) \prod_{j=1}^{6} p(w_j) \\
&= \arg\max_{\phi} \prod_{i=1}^{5} \mathcal{N}(t_i; h(r_i, c_i; \phi), 1) \prod_{j=1}^{6} \mathcal{N}(w_j; 0, 1) \\
&= \arg\min_{\phi} \sum_{i=1}^{5} (t_i - h(r_i, c_i; \phi))^2 + \underbrace{\sum_{j=1}^{6} (w_j)^2}_{\|\phi\|^2}
\end{aligned}$$

Here again, we have an objective $L'(\phi)$ to minimize, which we evaluate for each of the proposed sets.

$$\begin{aligned}
L'(\phi_A) &= L(\phi_A) + \sum_{j=1}^{6} (w_j)^2 = 29.75 + (-1.5)^2 + (1)^2 + \cdots = 88.25 \\
L'(\phi_B) &= 65.0 \\
L'(\phi_C) &= 257.0
\end{aligned}$$

The best set is ϕ_B.

(c) Discuss the procedure you would implement on a computer to find the optimal set of parameters, had the table above not been provided.

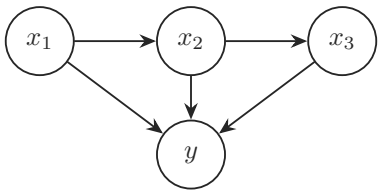
Solution. If the table is not provided, it is intractable to evaluate and compare all possible sets of parameters. However, we still want to minimize the MLE (or MAP) objective $L(\phi)$. We can improve a set of parameters ϕ by following the opposite of the objective's gradient $\nabla_\phi L(\phi)$, *i.e.* performing *gradient descent* steps

$$\phi \leftarrow \phi - \alpha \nabla_\phi L(\phi),$$

where α is the learning rate.

Exercise 2 Predict the leaderboard score

A leaderboard ranks language models by an overall score. The Bayesian network below represents how this score is computed. In this model, x_1, x_2 and x_3 respectively denote the scores obtained by a model on a validation set, on a public benchmark and on a held-out test set. The three evaluations share the same harness, and a model that does well on one tends to do well on the next. In particular, $x_2 \sim \mathcal{N}(a_1 x_1 + \mu_2, \sigma_2^2)$ and $x_3 \sim \mathcal{N}(a_2 x_2 + \mu_3, \sigma_3^2)$. Finally, $y \sim \mathcal{N}(a_3 x_1 + a_4 x_2 + a_5 x_3 + \mu_y, \sigma_y^2)$ stands for the overall score, which is a linear combination of the three scores plus some Gaussian noise due to rounding errors. Answer the following questions about this model.



(a) Assuming the parameters of the model are known, what is the expected value of y given x_1 and x_2?

Solution. Our task is to find the expectation

$$\mathop{\mathbb{E}}_{p(y|x_1,x_2)}[y] = \int y\, p(y \mid x_1, x_2)\, \mathrm{d}y.$$

We know that

$$p(y \mid x_1, x_2) = \int p(y \mid x_1, x_2, x_3)\, p(x_3 \mid x_1, x_2)\, \mathrm{d}x_3,$$

where $p(y \mid x_1, x_2, x_3)$ is given in the statement and $p(x_3 \mid x_1, x_2) = p(x_3 \mid x_2)$, since $x_3 \perp x_1 \mid x_2$. Both are linear Gaussian distributions. Therefore, we have

$$p(y \mid x_1, x_2) = \mathcal{N}\Big(y \mid a_3 x_1 + a_4 x_2 + a_5(a_2 x_2 + \mu_3) + \mu_y, (a_5 \sigma_3)^2 + \sigma_y^2\Big)$$

and, by definition of a Gaussian distribution,

$$\mathop{\mathbb{E}}_{p(y|x_1,x_2)}[y] = a_3 x_1 + (a_4 + a_5 a_2) x_2 + a_5 \mu_3 + \mu_y.$$

(b) Suppose now that the parameters are unknown, and that the scores x_3 on the held-out test set are not observed. Given a learning set $\mathbf{d} = \{(x_{i,1}, x_{i,2}, y_i)\}$ of N independent and identically distributed points, derive the maximum likelihood estimate of the parameters of $p(y \mid x_1, x_2)$.

Solution. We know that the distribution of y given x_1 and x_2 takes the form $\mathcal{N}(y \mid w_1 x_1 + w_2 x_2 + b, \sigma^2)$. Then, our task is to find the parameters $h = (w_1, w_2, b, \sigma)$ that maximize the likelihood of d, *i.e.* the *maximum likelihood estimation* (MLE)

$$\begin{aligned}
h_{\text{MLE}} &= \arg\max_h p(\mathbf{d} \mid h) \\
&= \arg\max_h \prod_i p(x_i, y_i \mid h) \\
&= \arg\max_h \log \prod_i p(x_i, y_i \mid h) \\
&= \arg\max_h \sum_i \log p(x_i, y_i \mid h) \\
&= \arg\max_h \sum_i \log p(y_i \mid h, x_i) + \log p(x_i) \\
&= \arg\max_h \sum_i \log p(y_i \mid h, x_i) \\
&= \arg\max_h \sum_i \log \left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(w^T x_i - y_i)^2}{2\sigma^2} \right) \right] \\
&= \arg\max_h \sum_i -\frac{1}{2}\log(2\pi\sigma^2) - \frac{(w^T x_i - y_i)^2}{2\sigma^2} \\
&= \arg\min_h \log\sigma^2 + \frac{1}{\sigma^2}\frac{1}{N}\sum_i (w^T x_i - y_i)^2,
\end{aligned}$$

where $x_i = \begin{pmatrix} x_{i,1} & x_{i,2} & 1 \end{pmatrix}^T$ and $w = \begin{pmatrix} w_1 & w_2 & b \end{pmatrix}^T$. In the last expression, we observe that the minimizer of the summation term is independent of σ. Therefore,

$$w_{\text{MLE}} = \arg\min_w \sum_i (w^T x_i - y_i)^2,$$

which exactly corresponds to a *linear regression* problem. Then, we find w_{MLE} by canceling the gradient with respect to w, *i.e.*

$$\begin{aligned}
0 &= \nabla_w \sum_i (w^T x_i - y_i)^2 \\
&= \nabla_w \sum_i (w^T x_i - y_i)(w^T x_i - y_i) \\
&= \nabla_w \sum_i \left((w^T x_i)^2 + y_i^2 - 2 w^T x_i y_i \right) \\
&= \nabla_w \left(w^T X^T X w + Y^T Y - 2 w^T X^T Y \right) \\
&= 2 X^T X w - 2 X^T Y
\end{aligned}$$

where $X = (x_i^T) \in \mathbb{R}^{N \times 3}$ and $Y = (y_i) \in \mathbb{R}^N$. Finally, we have

$$\begin{aligned}
0 &= X^T X w_{\text{MLE}} - X^T Y \\
\Leftrightarrow \quad w_{\text{MLE}} &= (X^T X)^{-1} X^T Y.
\end{aligned}$$

Afterwards, we find σ_{MLE} such that

$$\sigma_{\text{MLE}} = \arg\min_\sigma \log\sigma^2 + \frac{\text{MSE}}{\sigma^2}$$

$$= \sqrt{\text{MSE}},$$

where MSE denotes the *mean squared error*

$$\frac{1}{N}\sum_i ({w_{\text{MLE}}}^T x_i - y_i)^2.$$

By (a), the fitted parameters estimate combinations of those of the network, $w_1 = a_3$, $w_2 = a_4 + a_5 a_2$, $b = a_5\mu_3 + \mu_y$ and $\sigma^2 = a_5^2\sigma_3^2 + \sigma_y^2$. Without observations of x_3, the individual parameters $a_2, a_4, a_5, \mu_3, \mu_y, \sigma_3, \sigma_y$ cannot be recovered separately.

Exercise 3 Heteroscedastic linear regression

In linear regression with Gaussian noise of constant variance, $y_i \sim \mathcal{N}(w^T x_i, \sigma^2)$, the maximum likelihood estimate of the weights is $w_{\text{MLE}} = (X^T X)^{-1} X^T Y$, where the rows of X are the inputs x_i^T and $Y = (y_i)$. What does this estimate become if the noise is different for each sample, *i.e.* $y_i \sim \mathcal{N}(w^T x_i, \sigma_i^2)$ with known variances σ_i^2?

Solution. The log-likelihood of the weights is

$$L(w) = \sum_i \log \mathcal{N}(y_i \mid w^T x_i, \sigma_i^2) = -\sum_i \frac{(w^T x_i - y_i)^2}{2\sigma_i^2} + \text{const},$$

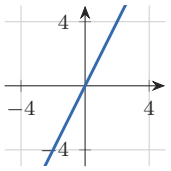
so that maximizing it minimizes a weighted sum of squares, each sample weighted by its precision $1/\sigma_i^2$. With $W = \text{diag}(1/\sigma_1^2, \ldots, 1/\sigma_N^2)$,

$$\nabla_w L(w) = -X^T W(Xw - Y) = 0 \quad \Leftrightarrow \quad w_{\text{MLE}} = (X^T W X)^{-1} X^T W Y.$$

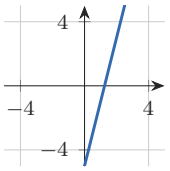
Noisy samples count less. With equal variances, W is proportional to the identity and the estimate of the homoscedastic case is recovered.

Exercise 4 Neural network representations (CS188, Spring 2024)

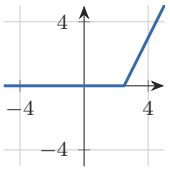
You are given a number of functions (a-h) of a single variable, x, which are graphed below. For each network structure proposed afterwards, indicate which functions they are able to represent. When possible, indicate appropriate values for the parameters.



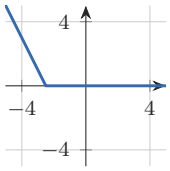
(a) $2x$



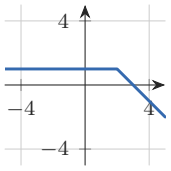
(b) $4x - 5$



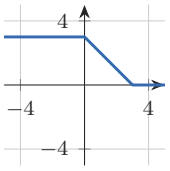
(c) $\begin{cases} 2x - 5 & x \geq 2.5 \\ 0 & x < 2.5 \end{cases}$



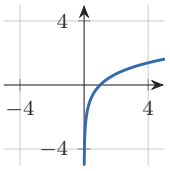
(d) $\begin{cases} -2x - 5 & x \leq -2.5 \\ 0 & x > -2.5 \end{cases}$



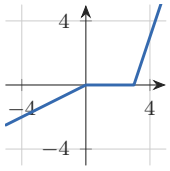
(e) $\begin{cases} -x + 3 & x \geq 2 \\ 1 & x < 2 \end{cases}$



(f) $\begin{cases} 3 & x \leq 0 \\ 3 - x & 0 < x \leq 3 \\ 0 & x > 3 \end{cases}$

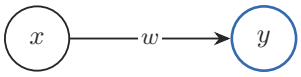


(g) $\log(x)$



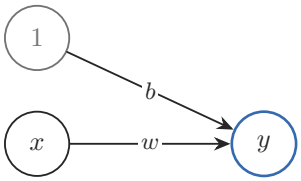
(h) $\begin{cases} 0.5x & x \leq 0 \\ 0 & 0 < x \leq 3 \\ 3x - 9 & x > 3 \end{cases}$

(a) We start with a linear transformation of the scalar input x, weight w, and output y, such that $y = wx$.



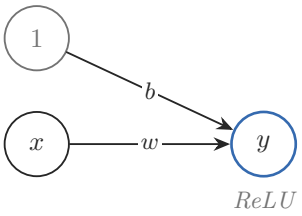
Solution. This network can only represent (a), with $w = 2$. Since there is no bias term, the line must pass through the origin.

(b) We then introduce a bias term b, such that $y = wx + b$.



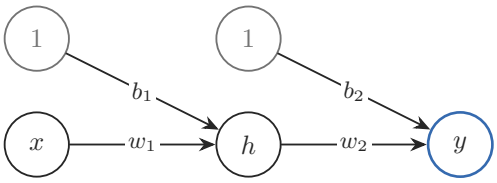
Solution. (a) with $w = 2$ and $b = 0$, and (b) with $w = 4$ and $b = -5$.

(c) We now introduce a non-linearity into the network, the ReLU $\text{ReLU}(x) = \max(0, x)$, so that $y = \text{ReLU}(wx + b)$.



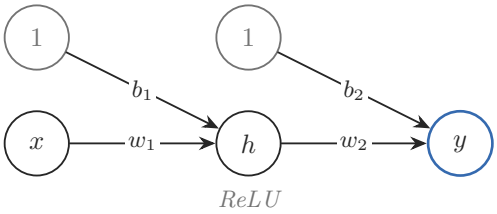
Solution. With the output coming directly from the ReLU, this cannot produce any values less than zero. It can produce (c) with $w = 2$ and $b = -5$, and (d) with $w = -2$ and $b = -5$.

(d) Now we consider neural networks with multiple affine transformations, $h = w_1 x + b_1$ and $y = w_2 h + b_2$, as depicted below.



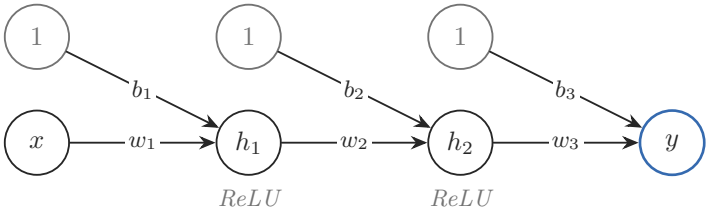
Solution. Applying multiple affine transformations (with no non-linearity in between) is not any more powerful than a single affine function: $w_2(w_1 x + b_1) + b_2 = w_2 w_1 x + w_2 b_1 + b_2$, so this is just an affine function with different coefficients. The functions we can represent are the same as in (b), if we choose $w_1 = w$, $w_2 = 1$, $b_1 = 0$, $b_2 = b$: (a) with $w_1 = 2$, $w_2 = 1$, $b_1 = 0$, $b_2 = 0$, and (b) with $w_1 = 4$, $w_2 = 1$, $b_1 = 0$, $b_2 = -5$.

(e) We now add a ReLU non-linearity to the network between the affine transformations: $h = \text{ReLU}(w_1 x + b_1)$ and $y = w_2 h + b_2$.



Solution. (c), (d), and (e). The affine transformation after the ReLU is capable of stretching (or flipping) and shifting the ReLU output in the vertical dimension. The parameters to produce these are: (c) with $w_1 = 2$, $b_1 = -5$, $w_2 = 1$, $b_2 = 0$, (d) with $w_1 = -2$, $b_1 = -5$, $w_2 = 1$, $b_2 = 0$, and (e) with $w_1 = 1$, $b_1 = -2$, $w_2 = -1$, $b_2 = 1$.

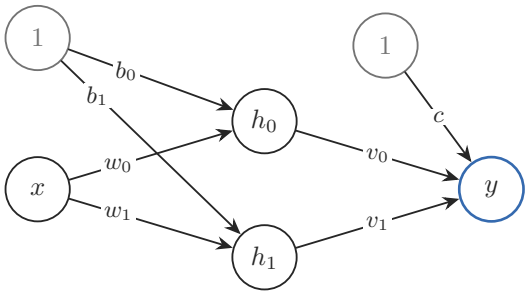
(f) Now we add another hidden layer to the network, as depicted below: $h_1 = \text{ReLU}(w_1 x + b_1)$, $h_2 = \text{ReLU}(w_2 h_1 + b_2)$ and $y = w_3 h_2 + b_3$. You do not have to guess parameters values anymore.



Solution. (c), (d), (e), and (f). The network can represent all the same functions as (e), since the second ReLU can let its input through ($w_2 = 1$ and $b_2 = 0$ when $h_1 \geq 0$). In addition it can represent (f): the first ReLU produces the first flat segment, the affine transformation flips and shifts the resulting curve, and the second ReLU produces the second flat segment, the final affine layer not doing anything.

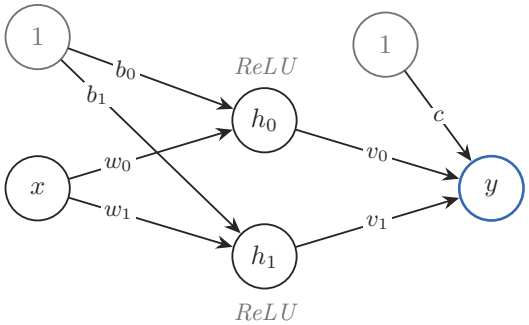
It cannot produce (h). With a single unit per layer, the output is a chain of functions of one variable: h_1 is flat on one side of its joint and sloped on the other; h_2 is constant wherever h_1 is, and on the other side applies a ReLU to an affine function, which can only add a flat part. The final layer scales and shifts the result. Such a chain has at most one segment with a nonzero slope, whereas (h) has two.

(g) We'd like to consider using a neural net with just one hidden layer, but larger. Let's first consider using just two affine functions, with no nonlinearity in between, $h_j = w_j x + b_j$ for $j = 0, 1$ and $y = v_0 h_0 + v_1 h_1 + c$. You do not have to guess parameters values anymore.



Solution. (a) and (b). With no non-linearity, this reduces to a single affine function (in the same way as (d)).

(h) Now we'll add a non-linearity between the two affine layers, $h_j = \text{ReLU}(w_j x + b_j)$ for $j = 0, 1$ and $y = v_0 h_0 + v_1 h_1 + c$, to produce the neural network below with a hidden layer of size 2. You do not have to guess parameters values anymore.



Solution. All functions except for (g). Setting $v_1 = 0$ gives back the network of (e), which produces (c), (d) and (e). The output is a weighted sum of two ReLUs, each flat on one side of its joint and sloped on the other, so that it can add two sloped segments or cancel one slope with the other. For instance, (a) is $2\text{ReLU}(x) - 2\text{ReLU}(-x)$, (b) is $\text{ReLU}(4x - 5) - \text{ReLU}(-4x + 5)$, (f) is $\text{ReLU}(3 - x) - \text{ReLU}(-x)$, and (h) is $-0.5\text{ReLU}(-x) + 3\text{ReLU}(x - 3)$.

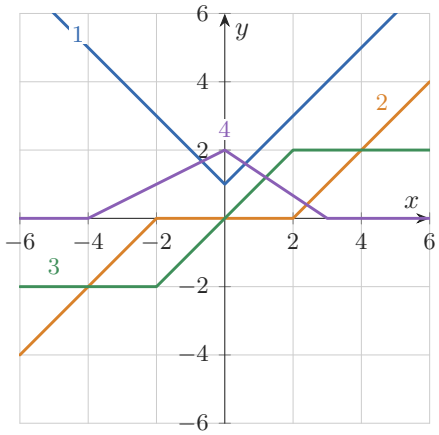
(i) Are there functions that cannot be represented by any of the proposed networks? If yes, explain why and what you would need to model them.

Solution. Yes, function (g), $\log(x)$. Every proposed network computes a piecewise linear function, and $\log(x)$ is not piecewise linear; it is moreover only defined for $x > 0$ and unbounded near 0.

By the universal approximation theorem, a network with one hidden layer of ReLU units, as in (h) but with more units, approximates any continuous function arbitrarily well on a closed and bounded interval. It can therefore approximate $\log(x)$ as closely as we want on any interval $[\epsilon, M]$ with $0 < \epsilon < M$, with more units for a better approximation, but not on the whole of $(0, \infty)$.

Exercise 5 Piecewise-linear functions

For each of the piecewise-linear functions below, write a function $y = f(x)$ as a composition of sums $(+, -)$, ReLU ($\text{ReLU}(x) = \max(x, 0)$) non-linearities, and real parameters (weights and biases) that exactly matches the function over $\mathbb{R}$.



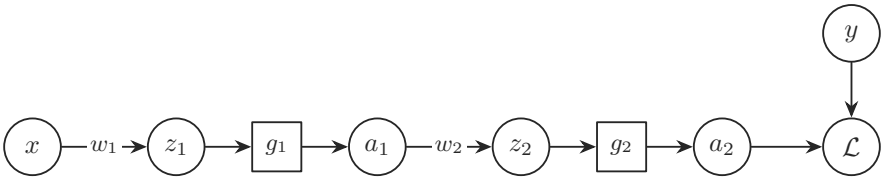
For example, $y = \text{ReLU}(x + 2) - \text{ReLU}(-2x)$ is a valid function.

Solution.

1. $y = \text{ReLU}(x) + \text{ReLU}(-x) + 1$
2. $y = \text{ReLU}(x - 2) - \text{ReLU}(-x - 2)$
3. $y = \text{ReLU}(-\text{ReLU}(-x + 2) + 4) - 2$
4. $y = \text{ReLU}(-\text{ReLU}(\frac{2}{3}x) - \text{ReLU}(-\frac{1}{2}x) + 2)$

Exercise 6 Binary classification (CS188, Spring 2024)

Consider the following simple neural network for binary classification. Here x is a single real-valued input feature with an associated class y (0 or 1). There are two weight parameters w_1 and w_2, and non-linearity functions g_1 and g_2. The network will output a value a_2 between 0 and 1, representing the probability of being in class 1. We will be using a loss function $\mathcal{L}$ to compare the prediction a_2 with the true class y.



(a) Write down the forward pass on this network, writing the output values for each node z_1, a_1, z_2 and a_2 in terms of the node's input values.

Solution.

$$\begin{aligned} z_1 &= w_1 x \\ a_1 &= g_1(z_1) = g_1(w_1 x) \\ z_2 &= w_2 a_1 = w_2 g_1(w_1 x) \\ a_2 &= g_2(z_2) = g_2(w_2 g_1(w_1 x)) \end{aligned}$$

(b) Derive the arguments of the loss $\mathcal{L}(a_2, y)$ in terms of the input x, weights w_i, and activation functions g_i.

Solution. The argument a_2 has been derived in the previous question. We have

$$a_2 = g_2(w_2 g_1(w_1 x)).$$

For the other input y, the latter is given as input of the problem. Indeed, in supervised learning, we assume access to a training dataset made of input-output pairs $\{(x_i, y_i)\}^N$ where y_i acts as a target for the network.

(c) Using the chain rule, differentiate $\mathcal{L}$ with respect to w_2. Write your expression as a product of partial derivatives at each node: *i.e.* the partial derivative of the node's output with respect to its inputs. (Hint: the series of expressions you wrote in (a) will be helpful; you may use any of those variables.)

Solution.

$$\frac{\partial \mathcal{L}}{\partial w_2} = \frac{\partial \mathcal{L}}{\partial a_2} \frac{\partial a_2}{\partial z_2} \frac{\partial z_2}{\partial w_2}$$

(d) Motivate a choice for the output activation function g_2 given the nature of the problem. Motivate also a loss function. Give the expression of the gradients of both functions with respect to their inputs.

Solution. The problem is a binary classification, and the output a_2 is the probability $P(Y = 1 \mid x)$, between 0 and 1. The output activation must therefore map $\mathbb{R}$ to $(0, 1)$: a typical choice that fulfills this requirement is the sigmoid function $\sigma(z) = \dfrac{1}{1 + e^{-z}}$, $\sigma : \mathbb{R} \to (0, 1)$. The gradient of the sigmoid with respect to its input is

$$\frac{\partial \sigma(z)}{\partial z} = \frac{e^{-z}}{(1 + e^{-z})^2} = \sigma(z)\sigma(-z) = \sigma(z)(1 - \sigma(z)).$$

For the loss function, we derive the cross-entropy from maximum likelihood estimation. The likelihood of a pair (x, y) can be derived from our model as

$$\begin{cases} \text{if } y = 1 \text{ then } P(Y = y \mid x) = \mathrm{NN}_\theta(x) \\ \text{if } y = 0 \text{ then } P(Y = y \mid x) = 1 - \mathrm{NN}_\theta(x) \end{cases}$$

where NN_θ is the neural network parameterized by $\theta = [w_1, w_2]$. Denoting $\mathrm{NN}_\theta(x) = a_2$, the likelihood of one pair is $a_2^y (1 - a_2)^{1-y}$, and its log-likelihood is

$$\log L(x, y; \theta) = y \log a_2 + (1 - y) \log(1 - a_2).$$

Since we want to maximize the log-likelihood, but minimize the loss, we can set the latter to

$$\mathcal{L}(a_2, y) = -y \log a_2 - (1-y)\log(1-a_2)$$

which corresponds to the binary cross-entropy loss. The gradient of the loss with respect to a_2 is

$$\frac{\partial \mathcal{L}(a_2, y)}{\partial a_2} = -\frac{y}{a_2} + \frac{(1-y)}{(1-a_2)} = \frac{a_2 - y}{a_2(1-a_2)}.$$

(e) We set the loss function to the binary cross-entropy

$$\mathcal{L}(a_2, y) = -y \log a_2 - (1-y)\log(1-a_2),$$

and g_1 and g_2 are both sigmoid functions $\sigma(z) = \frac{1}{1+e^{-z}}$. If you made this choice in the previous question, use your results to get the expression of $\frac{\partial \mathcal{L}}{\partial w_2}$. Otherwise, start by expressing $\frac{\partial \sigma(z)}{\partial z}$ and $\frac{\partial \mathcal{L}}{\partial a_2}$.

Solution. We still need $\frac{\partial z_2}{\partial w_2}$, which is

$$\frac{\partial z_2}{\partial w_2} = \frac{\partial}{\partial w_2}(w_2 a_1) = a_1.$$

Plugging all the partial derivatives into the expression found in (c), we have

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial w_2} &= \frac{a_2 - y}{a_2(1-a_2)} g_2(z_2)(1 - g_2(z_2))a_1 \\ &= \frac{a_2 - y}{a_2(1-a_2)} a_2(1-a_2)a_1 \\ &= (a_2 - y)a_1\end{aligned}$$

(f) Now use the chain rule to express $\frac{\partial \mathcal{L}}{\partial w_1}$ as a product of partial derivatives at each node of interest.

Solution.

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial \mathcal{L}}{\partial a_2}\frac{\partial a_2}{\partial z_2}\frac{\partial z_2}{\partial a_1}\frac{\partial a_1}{\partial z_1}\frac{\partial z_1}{\partial w_1}$$

(g) Finally, write $\dfrac{\partial \mathcal{L}}{\partial w_1}$ in terms of x, y, w_i, a_i, z_i.

Solution. We first need to compute

$$\frac{\partial a_1}{\partial z_1} = g_1(z_1)(1 - g_1(z_1)) = a_1(1-a_1)$$

similarly to $\frac{\partial a_2}{\partial z_2}$, and

$$\frac{\partial z_1}{\partial w_1} = \frac{\partial}{\partial w_1}(w_1 x) = x.$$

Then, we get

$$\frac{\partial \mathcal{L}}{\partial w_1} = (a_2 - y)w_2 a_1(1-a_1)x.$$

(h) What is the gradient descent update for w_1 with learning rate α in terms of the values computed above?

Solution.

$$w_1 \leftarrow w_1 - \alpha(a_2 - y)w_2 a_1(1 - a_1)x.$$

You see here the importance of the forward pass. Iteratively, when receiving the input x, we compute all the intermediate activations (here, a_1 and a_2) and the loss (given y). Then, we propagate backward (from the output to the input) the values following the chain rule for the gradient of the loss with respect to the parameters of interest. Finally, we update the parameters using the computed gradient.

Exercise 7 Training loop (Exam, January 2024)

Let us consider a neural network f with one hidden layer taking as input a scalar $x \in \mathbb{R}$ and producing as output a prediction $\hat{y} = f(x; \theta) \in \mathbb{R}$. The neural network is defined as

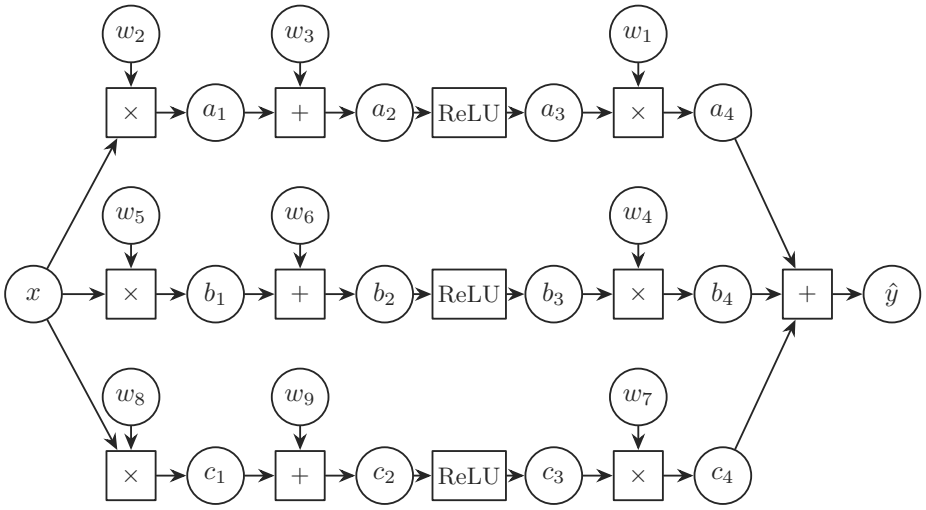
$$f(x; \theta) = w_1 \text{ReLU}(w_2 x + w_3) + w_4 \text{ReLU}(w_5 x + w_6) + w_7 \text{ReLU}(w_8 x + w_9),$$

where $\theta = (w_1, w_2, w_3, w_4, w_5, w_6, w_7, w_8, w_9)$ is the set of parameters and $\text{ReLU}(x) = \max(x, 0)$ is the rectified linear unit function.

(a) Draw the computation graph representing the neural network and the flow of information from inputs to outputs. Your diagram should be a directed graph that follows the following conventions:

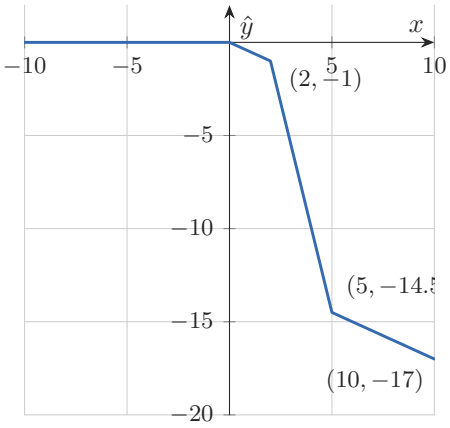
- circled nodes correspond to variables (input, output, parameters or intermediate variables),
- squared nodes correspond to primitive operations (addition, multiplication, ReLU) and produce an intermediate variable as output,
- directed edges correspond to the flow of information, from inputs to outputs.

Solution. The intermediate variables of the three hidden units are named a_k, b_k and c_k.



(b) For $\theta = (-1, \frac{1}{2}, 0, -4, 1, -2, 4, 1, -5)$, draw the function $\hat{y} = f(x;\theta)$ for $x \in [-10, 10]$.

Solution. The three units have their joints at $x = 0$, 2 and 5: $f(x;\theta) = -\text{ReLU}(x/2) - 4\text{ReLU}(x-2) + 4\text{ReLU}(x-5)$ is 0 for $x \leq 0$, then linear between the points $(0, 0)$, $(2, -1)$, $(5, -14.5)$ and $(10, -17)$.



(c) Using the data point $(x, y) = (10, -15)$ and the value of θ given above, we want to fine-tune the parameter w_8 of the neural network such that $f(x;\theta)$ produces a more accurate prediction of the target y.

i. Evaluate the squared error loss at the data point $(x, y) = (10, -15)$ and the current value of θ.
Solution.

$$\hat{y} = f(10;\theta) = -17$$
$$\mathcal{L}(\hat{y} = -17, y = -15) = (-17 - (-15))^2 = 4$$

ii. Derive an expression for the derivative of the squared error loss with respect to w_8.
Solution. The loss is

$$\mathcal{L}(\hat{y}, y) = (\hat{y} - y)^2.$$

We compute the gradient

$$\frac{\partial \mathcal{L}}{\partial w_8} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial c_4} \frac{\partial c_4}{\partial c_3} \frac{\partial c_3}{\partial c_2} \frac{\partial c_2}{\partial c_1} \frac{\partial c_1}{\partial w_8}$$
$$= 2(\hat{y} - y) w_7 \frac{\partial c_3}{\partial c_2} x.$$

We need to compute forward activations to compute the gradient value. $\frac{\partial c_3}{\partial c_2}$ will be 1 if $c_2 > 0$ else 0 by definition of the ReLU. We have

$$\hat{y} = -17$$
$$y = -15$$
$$w_7 = 4$$
$$x = 10$$
$$c_2 = w_8 x + w_9 = 5 \Rightarrow \frac{\partial c_3}{\partial c_2} = 1$$

which gives a gradient equal to -160.

iii. Update the parameter w_8 using one step of gradient descent with a learning rate $\alpha = 0.0005$.
Solution. The gradient step is given by

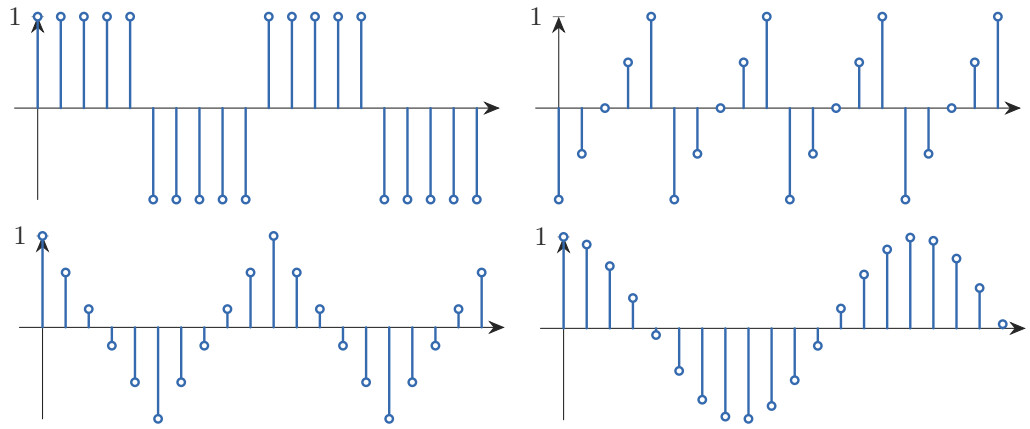
$$w_8 \leftarrow w_8 - \alpha \frac{\partial \mathcal{L}}{\partial w_8}$$

which gives an updated w_8 equal to 1.08.

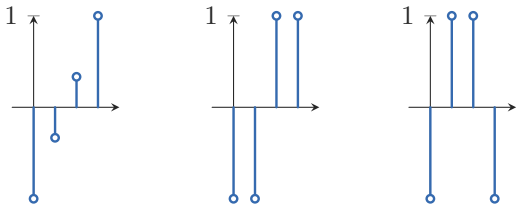
iv. Verify if the value of the loss function has decreased after the update.
Solution. With the updated w_8, we have $\hat{y} = -13.8$ which gives a loss equal to 1.44. The previous loss value was 4, we indeed decreased the loss.

Exercise 8 1d convolution

Consider the following four signals $x \in \mathbb{R}^{20}$, numbered 1 to 4 from left to right and top to bottom,



and the following three convolution kernels $\omega \in \mathbb{R}^4$.



$(-1, -\frac{1}{3}, \frac{1}{3}, 1)$ $(-1, -1, 1, 1)$ $(-1, 1, 1, -1)$

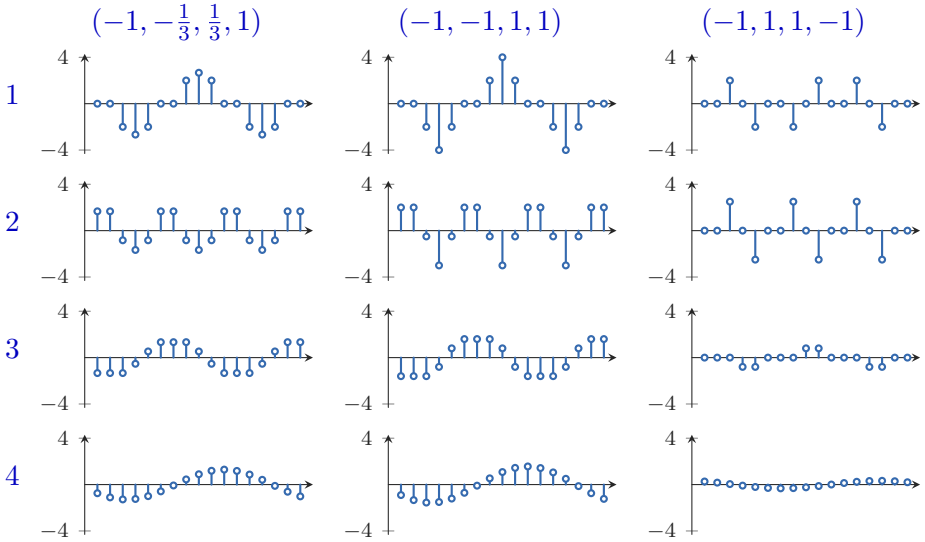
(a) Compute the convolution $x \circledast \omega$ of the first signal (the square wave) with each kernel, without padding, so that each output has $20 - 4 + 1 = 17$ values.

Solution. With $(x \circledast \omega)[i] = \sum_{m=0}^{3} x_{i+m}\omega_m$ for $i = 0, \ldots, 16$, the square wave (+1 for five samples, then −1 for five samples) gives

$$(-1, -\tfrac{1}{3}, \tfrac{1}{3}, 1) : (0, 0, -2, -\tfrac{8}{3}, -2, 0, 0, 2, \tfrac{8}{3}, 2, 0, 0, -2, -\tfrac{8}{3}, -2, 0, 0),$$
$$(-1, -1, 1, 1) : (0, 0, -2, -4, -2, 0, 0, 2, 4, 2, 0, 0, -2, -4, -2, 0, 0),$$
$$(-1, 1, 1, -1) : (0, 0, 2, 0, -2, 0, 0, -2, 0, 2, 0, 0, 2, 0, -2, 0, 0).$$

(b) Without computing every value, sketch the convolution of the three other signals with each kernel. What does each kernel detect?

Solution. The outputs below share the vertical scale $[-4, 4]$, one row per signal and one column per kernel (the first row is (a)).



The first kernel is a smoothed derivative: its output is positive where the signal increases, negative where it decreases and zero where it is constant, so that the triangle wave gives a rounded square wave and the cosine a sine. The second compares the two halves of its window: it also measures the slope, but responds most strongly to jumps, with peaks of ± 4 at the edges of the square wave. The third compares the centre of its window with its ends: it is zero wherever the signal is linear (the flat parts of the square wave, the ramps of the sawtooth and of the triangle), positive at local maxima and negative at local minima, so that it detects corners; on the cosine, whose curvature is proportional to the signal itself, it returns a small copy of the signal.

Exercise 9 2d convolution

(a) Represent the 2D convolution $x \circledast \omega$ of the following greyscale image x, where a value of 0 corresponds to a white pixel and a value of 1 corresponds to a black pixel, with the following convolution kernel ω (the box marks the window of the first output value).

$x \in \mathbb{R}^{5\times 5}$

0.1	0.3	0.6	0.9	0.4
0.2	0.5	0.8	0.7	0.3
0.4	0.7	1.0	0.8	0.5
0.2	0.6	0.9	0.7	0.3
0.1	0.4	0.7	0.5	0.2

$\omega \in \mathbb{R}^{2\times 3}$

0.2	0.6	0.9
0.4	0.8	0.3

Is the result dependent on the choice of coordinate system (*i.e.* on which directions are considered the "i" and "j" directions in $x[j, i]$ and $\omega[j, i]$)?

Solution. The result is equivariant to the choice of coordinate system. The resulting feature map, as drawn on the sheet of paper, remains the same. Only the interpretation of which

pixel is represented by $(x \circledast \omega)[j, i]$ changes.

$x \circledast \omega \in \mathbb{R}^{4\times 3}$

1.46	2.28	1.99
2.08	2.53	2.04
2.23	2.63	2.14
1.78	2.16	1.61

(b) The image x is now a colour image, $x \in \mathbb{R}^{3\times 4\times 4}$, with three channels (one for the red colour, one for the green colour and one for the blue colour):

$x[0, :, :]$ (red)

0.1	0.3	0.6	0.9
0.2	0.5	0.8	0.7
0.4	0.7	1.0	0.8
0.2	0.6	0.9	0.7

$x[1, :, :]$ (green)

0.2	0.2	0.2	1.0
0.2	0.2	0.2	1.0
0.2	0.2	0.2	1.0
0.2	0.2	0.2	1.0

$x[2, :, :]$ (blue)

0.2	0.2	0.2	0.1
0.2	0.2	0.2	0.1
0.2	0.2	0.2	0.1
0.2	0.2	0.2	0.1

Represent the 2D convolution $x \circledast \omega$ with the following convolution kernel $\omega \in \mathbb{R}^{3\times 2\times 3}$:

$\omega[0, :, :]$

0.2	0.6	0.9
0.4	0.8	0.3

$\omega[1, :, :]$

0.1	0.1	0.1
0.1	0.1	0.1

$\omega[2, :, :]$

0.2	0.2	0.2
0.2	0.2	0.2

Is the result dependent on the choice of coordinate system? What happens if the channels of x are shuffled?

Solution. The dimension corresponding to the channels cannot be interchanged with those corresponding to the height or width of the image while preserving the validity and correctness of the operation: the operation is a 2D convolution applied to a three-dimensional tensor, which sums over the channels. Shuffling the channels of x alone changes the result, since each channel then meets another slice of the kernel; permuting the channels of x and of ω in the same way leaves it unchanged, since it only reorders the sum over c. For the 2D coordinate system used to describe the spatial dimensions of the images at each channel (width and height), the same result as in (a) holds.

$x \circledast \omega \in \mathbb{R}^{3\times 2}$

1.82	2.76
2.44	3.01
2.59	3.11

Exercise 10 Problem statement, output layer and loss function

A dataset $\mathbf{d}$ consists of $N = 500$ images of cats and dogs, with exactly one cat or one dog appearing in each image. Each image has a spatial resolution of 32×32 pixels and is encoded in RGB format using three channels of real-valued entries. The first channel represents the red component of the image, the second channel represents the green component, and the third channel represents the blue component.

You would like to formulate a supervised learning problem in order to develop a deep learning model suited to a given task.

You are interested in three different tasks:

1. Assign a beauty score (a real-valued quantity) to a given image.
2. Determine whether the image contains a cat or a dog.
3. Determine whether each pixel of an image belongs to a cat, a dog or the background.

For each problem, you want to identify a probabilistic model in the form $p_\theta(y \mid x)$ from a dataset that you will construct:

$$\mathbf{d} = \{(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)\}.$$

(a) For each task, define the random variables X and Y, specify their respective dimensions, and explain what θ represents.

Solution. For each task, θ represents the parameters of the model being developed. X is the random variable of the image given as input to the model, $X \in \mathbb{R}^{3\times32\times32}$.

For the first task, Y is the random variable of the beauty score associated with an image, $Y \in \mathbb{R}$.

For the second task, Y is a binary random variable indicating whether the image contains a dog (let's say $Y = 1$ in this case) or a cat ($Y = 0$): $Y \in \{0, 1\}$.

For the third case, Y is the 32×32 array of categorical random variables $P_{u,v}$, one for each pixel of the image X:

$$Y = (P_{u,v})_{0\leq u,v\leq 31}.$$

Each $P_{u,v}$ indicates whether pixel (u, v) belongs to a cat, a dog, or the background: $P_{u,v} \in \{\text{cat}, \text{dog}, \text{background}\}$.

Once the task has been formulated, you construct the dataset $\mathbf{d}$ by annotating each of the 500 images, thereby obtaining a sample $y_i \sim Y$ associated with a sampled image $x_i \sim X$.

(b) Show that finding the parameters θ that maximize the likelihood of your model over a sample of $\mathbf{d}$,

$$p\left((x_i, y_i) \mid \theta\right),$$

is equivalent to finding the parameters θ that maximize the probability of the modelled conditional distribution over this sample,

$$p\left(y_i \mid x_i, \theta\right).$$

Solution.

$$\begin{aligned}
\arg\max_\theta \left\{p\left((x_i, y_i) \mid \theta\right)\right\} &= \arg\max_\theta \left\{p\left(y_i \mid x_i, \theta\right) p(x_i \mid \theta)\right\} \\
&= \arg\max_\theta \left\{p\left(y_i \mid x_i, \theta\right) p(x_i)\right\} \\
&\qquad \text{(As the model is over } Y \mid X\text{, not } X\text{.)} \\
&= \arg\max_\theta \left\{p\left(y_i \mid x_i, \theta\right)\right\}
\end{aligned}$$

After reviewing the literature, you choose a model for $p_\theta(y \mid x) = p(y \mid x, \theta)$.
For the first task, you assume that a suitable model is to model $Y \mid X$ as a Gaussian distribution with homoscedastic variance. For the second task, a suitable model is a Bernoulli distribution. For the third task, a suitable model is a categorical distribution.

(c) For each task, propose a suitable loss function to train your neural network. Propose also an output layer for your neural network in the form

$$\mathbf{y} = \sigma_{\text{out}} \left(\mathbf{W}_y \mathbf{h} + \mathbf{b}_y \right).$$

Let $\mathbf{h}$ denote the output of the layer immediately preceding the output layer, with dimension d_o ($\mathbf{h} \in \mathbb{R}^{d_o}$). Specify the dimensions of $\mathbf{y}$, $\mathbf{W}_y$, and $\mathbf{b}_y$ and give an example of a suitable activation function σ_{out} for each task.
Hint: For each task, which parameter of the distribution $p_\theta(y \mid x)$ is estimated by your neural network?

Solution. For the first task, the statistical model is:

$$Y \mid X \sim \mathcal{N}\left(\mu(X), \sigma^2\right)$$
$$\rightarrow p(y \mid x) \propto \exp\left(-\frac{1}{2}\frac{(y - \mu(x))^2}{\sigma^2}\right)$$

The neural network estimates the mean of the Gaussian distribution and outputs $\hat{\mu}(X; \theta)$.
For the second task:

$$Y \mid X \sim \text{Bern}\left(q(X)\right)$$
$$\rightarrow p(y \mid x) \propto q(x)^y (1 - q(x))^{(1-y)}$$

The neural network estimates the probability that the image contains a dog and outputs $\hat{q}(X; \theta)$.
For the third task, number the classes 0 (dog), 1 (cat) and 2 (background). Each pixel (u, v) has its own categorical distribution, the pixels being independent given the image:

$$P_{u,v} \mid X \sim \text{Categ}\left(q_{0,uv}(X), q_{1,uv}(X), q_{2,uv}(X)\right)$$
$$\rightarrow p(y \mid x) = \prod_{u=0}^{31} \prod_{v=0}^{31} \prod_{c=0}^{2} q_{c,uv}(x)^{y_{c,uv}},$$

with $\sum_c q_{c,uv}(x) = 1$, and where $y_{c,uv}$ is the one-hot encoding of the label: 1 if pixel (u, v) belongs to class c, 0 otherwise. The neural network estimates the probabilities that each pixel belongs to a dog, a cat, or the background: it outputs $\hat{q}_{c,uv}(X; \theta)$ for every class c and pixel (u, v).
From the statistical model, and assuming that the samples (x_i, y_i) are independent and identically distributed, a loss function can be derived to maximize the likelihood of the

model given the observed dataset $\mathbf{d}$:

$$\begin{aligned}
\theta_{\text{MLE}} &= \arg\max_{\theta} p(\mathbf{d} \mid \theta) \\
&= \arg\max_{\theta} p((x_1, y_1), \ldots, (x_N, y_N) \mid \theta) \\
&= \arg\max_{\theta} \prod_{i=1}^{N} p(x_i, y_i \mid \theta) \\
&= \arg\max_{\theta} \prod_{i=1}^{N} p(y_i \mid x_i, \theta) \\
&= \arg\max_{\theta} \sum_{i=1}^{N} \log(p(y_i \mid x_i, \theta)) \\
&= \arg\min_{\theta} \sum_{i=1}^{N} -\log(p(y_i \mid x_i, \theta)) \\
&= \arg\min_{\theta} \mathcal{L}(\theta, \mathbf{d})
\end{aligned}$$

For the first task with Gaussian likelihood, the loss is the mean squared error:

$$\mathcal{L}(\theta, \mathbf{d}) = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{\mu}(x_i; \theta))^2$$

For the second task with a Bernoulli likelihood, the loss is the binary cross-entropy:

$$\mathcal{L}(\theta, \mathbf{d}) = -\frac{1}{N} \sum_{i=1}^{N} \left[y_i \log(\hat{q}(x_i; \theta)) + (1 - y_i) \log(1 - \hat{q}(x_i; \theta)) \right]$$

For the third task with categorical likelihood, the loss is the cross-entropy:

$$\mathcal{L}(\theta, \mathbf{d}) = -\frac{1}{N} \sum_{i=1}^{N} \frac{1}{32 \times 32} \sum_{u=0}^{31} \sum_{v=0}^{31} \sum_{c=0}^{2} y_{i,c,uv} \log(\hat{q}_{c,uv}(x_i; \theta)),$$

where $y_{i,c,uv}$ is the one-hot encoding of the label of pixel (u, v) in the i-th image.
Regarding the output layer, the linear projection

$$\mathbf{W}_y \mathbf{h} + \mathbf{b}_y$$

must map the output $\mathbf{h}$ of the preceding layer of the neural network to the desired output dimension.
For the first and the second task, $\mathbf{y} \in \mathbb{R}$, with

$$\mathbf{W}_y \in \mathbb{R}^{1 \times d_o}, \quad \mathbf{b}_y \in \mathbb{R}.$$

For the third task, the output holds one value per class and pixel, $3 \times 32 \times 32 = 3072$ values read as a tensor $\mathbf{y} \in \mathbb{R}^{3 \times 32 \times 32}$, with

$$\mathbf{W}_y \in \mathbb{R}^{3072 \times d_o}, \quad \mathbf{b}_y \in \mathbb{R}^{3072}.$$

The activation function σ_{out} should map the projected output to the valid range of values for $\mathbf{y}$. For the first task, we can choose the identity:

$$\sigma_{\text{out}}(z) = z.$$

For the second task, we can choose a sigmoid activation to map the projected values between 0 and 1:

$$\sigma_{\text{out}}(z) = \frac{1}{1+\exp(-z)}.$$

For the third task, we can choose the softmax over the three classes of each pixel, to turn the logits $z_{c,uv}$ into probabilities:

$$\hat{q}_{c,uv} = \frac{\exp(z_{c,uv})}{\sum_{c'=0}^{2}\exp(z_{c',uv})} \quad \text{for } c = 0,1,2 \text{ and every pixel } (u,v).$$

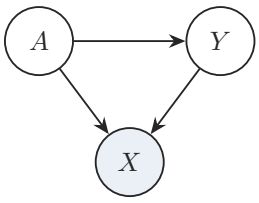
You want to solve a new task: estimate the age of the dog or the cat in the image. To simplify the problem, you consider the age as an unrestricted real-valued quantity. After some reflection, it becomes clear that directly inferring the age from the image is not a good idea. If the type of animal (cat or dog) were first estimated, it would be much easier to estimate the animal's age from the image and this information.

(d) Formulate the task in the context of supervised learning and explain all the elements appearing in $p_\theta(y \mid x)$. What mathematical quantity is estimated by the neural network?

Solution. $y \sim Y$ is the age of the animal in the picture ($Y \in \mathbb{R}$).
$x \sim X$ is the picture ($X \in \mathbb{R}^{3\times32\times32}$).
θ is the parameters of the model.
The problem is a regression problem. The neural network should give an estimate of $\mathbb{E}\{Y \mid X\}$.

(e) Draw a Bayesian network modelling the image, the type of animal and its age. Find a new expression for $p(y \mid x)$ that makes the effect of the type of the animal in the image explicit.

Solution. Let A denote the random variable describing the type of animal present in the image (let's say $A = 1$ if it is a dog). The type of the animal influences its age (cats and dogs do not have the same life span), and the animal and its age determine what the image shows. The image X is observed.



Any other ordering of the three variables, such as $X \to A$, $X \to Y$, $A \to Y$, gives a valid factorization of the same joint distribution, and the expression below holds whatever the network:

$$\begin{aligned} p(y \mid x) &= \sum_a p(y \mid A = a, x)p(A = a \mid x) \\ &= p(y \mid A = 1, x)p(A = 1 \mid x) + p(y \mid A = 0, x)(1 - p(A = 1 \mid x)) \end{aligned}$$

(f) Consider a large neural network designed to solve the task, divided into multiple submodules, each implemented as a separate neural network. Use the image classification model developed above as one of these submodules. Define the inputs and outputs of the additional submodules required to solve the task. Relate the resulting architecture to the mathematical expression derived above. Which submodule estimates each of the distributions appearing in this expression?

Hint: Derive an expression for $\mathbb{E}\{Y \mid X\}$ using the Bayesian network, explicitly accounting for the animal type A.

Solution. The neural network gives an estimate of the conditional expectation:

$$\begin{aligned}\mathbb{E}\{Y \mid X\} &= \mathbb{E}\{\mathbb{E}\{Y \mid X, A\} \mid X\} \\ &= \sum_a \mathbb{E}\{Y \mid X, A = a\} p(A = a \mid X) \\ &= \mathbb{E}\{Y \mid X, A = 1\} p(A = 1 \mid X) + \mathbb{E}\{Y \mid X, A = 0\}(1 - p(A = 1 \mid X))\end{aligned}$$

The expression can also be obtained directly by applying the expectation operator $\mathbb{E}_{Y|X}\{\cdot\}$ to both sides of the expression derived in (e).

The expected age is therefore a weighted mean of the age inferred under the assumption that the image contains a dog and the age inferred under the assumption that it contains a cat. Each age is weighted by the corresponding probability.

The first submodule is the image classification model developed previously. With parameters ϕ, it takes an image x as input and estimates the probability that the image contains a dog:

$$\hat{q}(x; \phi) \approx p(A = 1 \mid x) = 1 - p(A = 0 \mid x)$$

The second submodule, with parameters ξ, takes the image x as input and estimates the expected age of the animal under the assumption that it is a dog:

$$\hat{\mu}_d(x; \xi) \approx \mathbb{E}\{Y \mid x, A = 1\}$$

A third submodule, with parameters ψ, takes the image X as input and estimates the expected age under the assumption that it is a cat:

$$\hat{\mu}_c(x; \psi) \approx \mathbb{E}\{Y \mid x, A = 0\}$$

The final estimated age can then be reconstructed from the outputs of these submodules.

Exercise 11 Shallow network expressiveness (Exam, August 2026, adapted)
We consider a neural network for binary classification that takes as input a 2D point $(x_1, x_2) \in \mathbb{R}^2$ and outputs a probability $\hat{y} = f(x_1, x_2) \in [0, 1]$. The network has one hidden layer with two units and is defined as

$$\begin{aligned}h_0 &= \mathrm{ReLU}(x_1 - 1), \\ h_1 &= \mathrm{ReLU}(x_2 - 2), \\ \hat{y} &= \sigma(h_0 + h_1 - 2),\end{aligned}$$

where

$$\mathrm{ReLU}(z) = \max(0, z)$$

is the rectified linear unit and

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

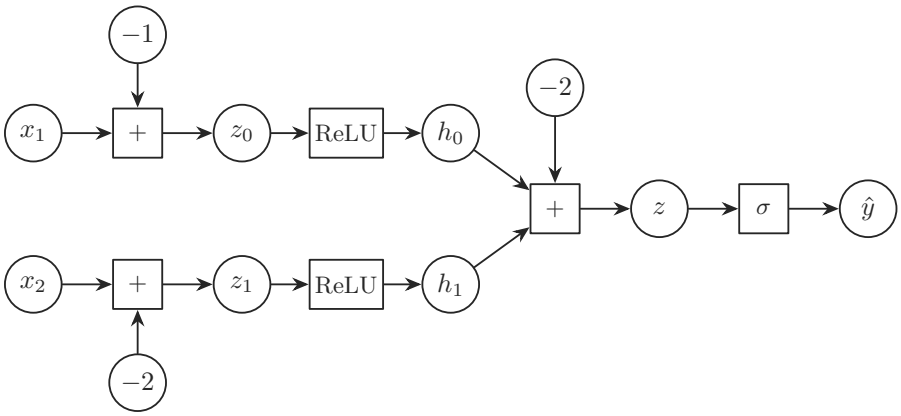
is the sigmoid function.

The predicted class is

$$\hat{c} = \begin{cases} 1, & \text{if } \hat{y} \geq 0.5, \\ 0, & \text{otherwise.} \end{cases}$$

(a) Draw the computation graph representing the neural network. Use circled nodes for variables (inputs, outputs, and intermediate values) and squared nodes for operations (addition, ReLU, and sigmoid).

Solution.



(b) The ReLU activation functions divide the input space into several regions, depending on whether each hidden unit is active ($h_j > 0$) or inactive ($h_j = 0$).

i. Identify and characterize these regions in terms of conditions on x_1 and x_2.

Solution. The two ReLU units are characterized by

$$\begin{aligned} x_1 > 1 &\implies h_0 = x_1 - 1 > 0, \\ x_1 \leq 1 &\implies h_0 = 0, \end{aligned}$$

and

$$\begin{aligned} x_2 > 2 &\implies h_1 = x_2 - 2 > 0, \\ x_2 \leq 2 &\implies h_1 = 0. \end{aligned}$$

Therefore, four regions can be distinguished:

1. $$\begin{cases} x_1 > 1 \\ x_2 > 2 \end{cases} \implies \begin{cases} h_0 = x_1 - 1 \\ h_1 = x_2 - 2 \end{cases}$$
2. $$\begin{cases} x_1 > 1 \\ x_2 \leq 2 \end{cases} \implies \begin{cases} h_0 = x_1 - 1 \\ h_1 = 0 \end{cases}$$
3. $$\begin{cases} x_1 \leq 1 \\ x_2 > 2 \end{cases} \implies \begin{cases} h_0 = 0 \\ h_1 = x_2 - 2 \end{cases}$$
4. $$\begin{cases} x_1 \leq 1 \\ x_2 \leq 2 \end{cases} \implies \begin{cases} h_0 = 0 \\ h_1 = 0 \end{cases}$$

ii. For each region, give a simplified expression for the output probability $\hat{y}$ as a function of x_1 and x_2.

Solution.

1. For $x_1 > 1$ and $x_2 > 2$,

$$\begin{aligned} \hat{y} &= \sigma(h_0 + h_1 - 2) \\ &= \sigma((x_1 - 1) + (x_2 - 2) - 2) \\ &= \sigma(x_1 + x_2 - 5) \\ &= \boxed{\frac{1}{1 + e^{-(x_1 + x_2 - 5)}}}. \end{aligned}$$

2. For $x_1 > 1$ and $x_2 \leq 2$,

$$\begin{aligned}\hat{y} &= \sigma(h_0 + h_1 - 2) \\ &= \sigma((x_1 - 1) + 0 - 2) \\ &= \sigma(x_1 - 3) \\ &= \boxed{\frac{1}{1 + e^{-(x_1 - 3)}}}.\end{aligned}$$

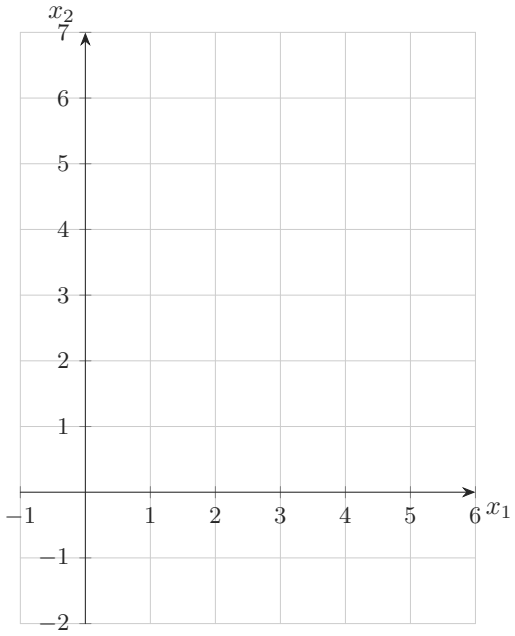
3. For $x_1 \leq 1$ and $x_2 > 2$,

$$\begin{aligned}\hat{y} &= \sigma(h_0 + h_1 - 2) \\ &= \sigma(0 + (x_2 - 2) - 2) \\ &= \sigma(x_2 - 4) \\ &= \boxed{\frac{1}{1 + e^{-(x_2 - 4)}}}.\end{aligned}$$

4. For $x_1 \leq 1$ and $x_2 \leq 2$,

$$\begin{aligned}\hat{y} &= \sigma(0 + 0 - 2) \\ &= \sigma(-2) \\ &= \boxed{\frac{1}{1 + e^{2}}}.\end{aligned}$$

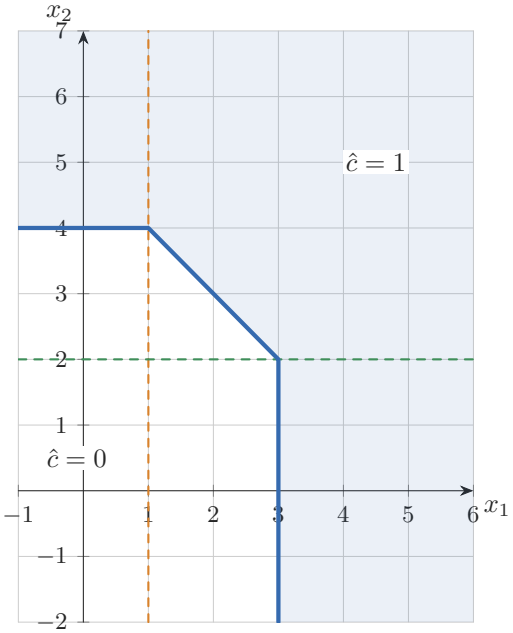
(c) Plot the decision boundary $\hat{y} = 0.5$ of the classifier on the grid below. Clearly indicate which region(s) are classified as class 1 and which are classified as class 0.



Solution. Since the sigmoid is increasing and $\sigma(0) = 0.5$, we have $\hat{y} \geq 0.5$ if and only if $h_0 + h_1 - 2 \geq 0$. In each region of (b):

1. for $x_1 > 1$ and $x_2 > 2$, $\hat{c} = 1$ if and only if $x_1 + x_2 \geq 5$;
2. for $x_1 > 1$ and $x_2 \leq 2$, $\hat{c} = 1$ if and only if $x_1 \geq 3$;
3. for $x_1 \leq 1$ and $x_2 > 2$, $\hat{c} = 1$ if and only if $x_2 \geq 4$;
4. for $x_1 \leq 1$ and $x_2 \leq 2$, $\hat{y} = \sigma(-2) < 0.5$, so $\hat{c} = 0$ everywhere.

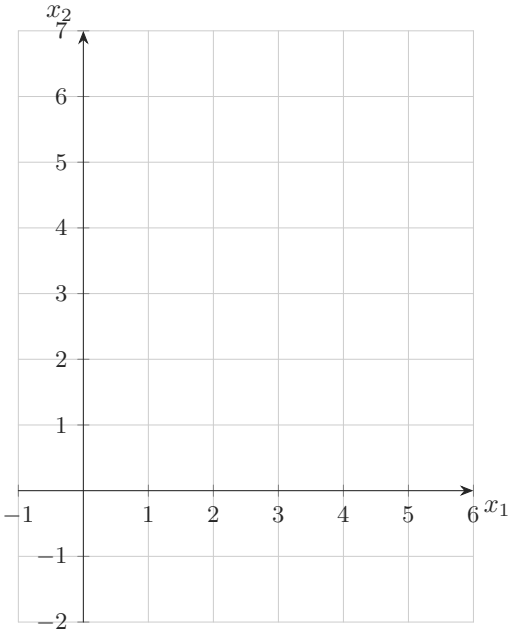
The decision boundary is the broken line from $(-\infty, 4)$ to $(1, 4)$, $(3, 2)$ and $(3, -\infty)$; class 1 (shaded) lies above and to the right of it, class 0 below and to the left. The joints of the hidden units are dashed.



(d) We now consider a modified network with a third hidden unit:

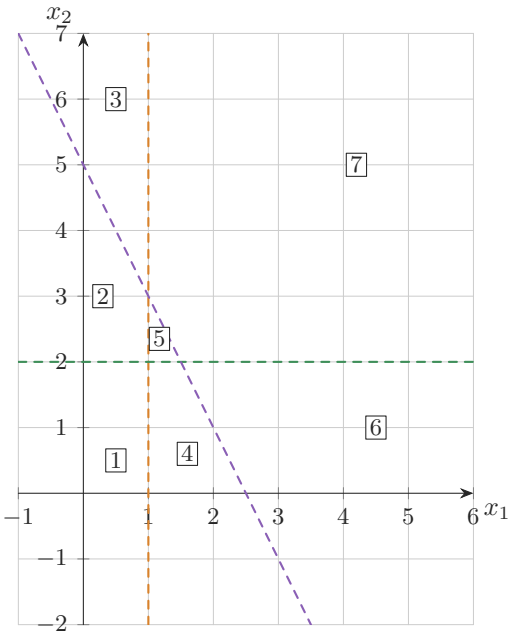
$$
\begin{aligned}
h_0 &= \mathrm{ReLU}(x_1 - 1), \\
h_1 &= \mathrm{ReLU}(x_2 - 2), \\
h_2 &= \mathrm{ReLU}(-2x_1 - x_2 + 5), \\
\hat{y} &= \sigma(h_0 + h_1 + h_2 - 2).
\end{aligned}
$$

On the grid below, plot the new decision boundary and compare it to the original one. What is the effect of adding h_2?



Solution. The new unit is active when $2x_1 + x_2 < 5$. The three joints $x_1 = 1$, $x_2 = 2$ and

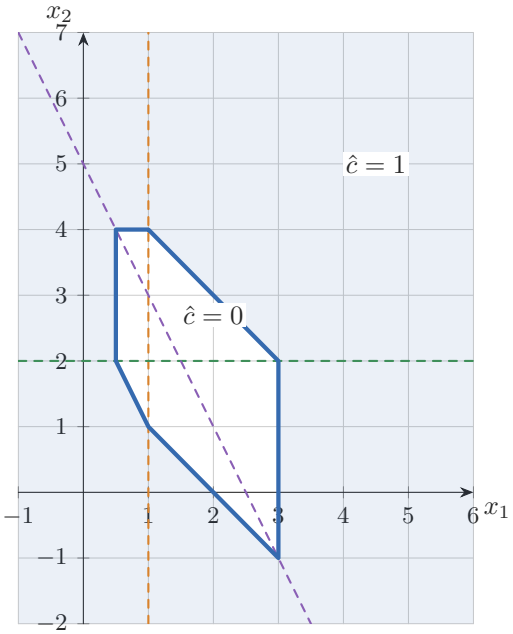
$2x_1 + x_2 = 5$ cut the plane into seven linear regions; the eighth activation pattern, with all units inactive, cannot occur, since $x_1 \leq 1$ and $x_2 \leq 2$ give $2x_1 + x_2 \leq 4 < 5$.



With $z = h_0 + h_1 + h_2 - 2$, the class is 1 where $z \geq 0$:

1. $x_1 \leq 1$, $x_2 \leq 2$ (h_2 only): $z = -2x_1 - x_2 + 3$, class 1 where $2x_1 + x_2 \leq 3$;
2. $x_1 \leq 1$, $x_2 > 2$, $2x_1 + x_2 < 5$ (h_1, h_2): $z = -2x_1 + 1$, class 1 where $x_1 \leq \frac{1}{2}$;
3. $x_1 \leq 1$, $x_2 > 2$, $2x_1 + x_2 \geq 5$ (h_1 only): $z = x_2 - 4$, class 1 where $x_2 \geq 4$;
4. $x_1 > 1$, $x_2 \leq 2$, $2x_1 + x_2 < 5$ (h_0, h_2): $z = -x_1 - x_2 + 2$, class 1 where $x_1 + x_2 \leq 2$;
5. $x_1 > 1$, $x_2 > 2$, $2x_1 + x_2 < 5$ (all three): $z = -x_1 < 0$, class 0 everywhere;
6. $x_1 > 1$, $x_2 \leq 2$, $2x_1 + x_2 \geq 5$ (h_0 only): $z = x_1 - 3$, class 1 where $x_1 \geq 3$;
7. $x_1 > 1$, $x_2 > 2$, $2x_1 + x_2 \geq 5$ (h_0, h_1): $z = x_1 + x_2 - 5$, class 1 where $x_1 + x_2 \geq 5$.

Class 0 is now the bounded hexagon with vertices $(1, 1)$, $(\frac{1}{2}, 2)$, $(\frac{1}{2}, 4)$, $(1, 4)$, $(3, 2)$ and $(3, -1)$, and everything else is class 1 (shaded), including the origin, where $z = 3$.



Where h_2 is inactive ($2x_1 + x_2 \geq 5$, regions 3, 6 and 7), the boundary is unchanged. Where it is active, it adds $-2x_1 - x_2 + 5$ to z and changes the decision: the region of class 0, which was unbounded and contained the origin, becomes a bounded hexagon. One more unit adds one more joint, and with it more linear regions and a more complex decision boundary.

Exercise 12 Neural network parameters

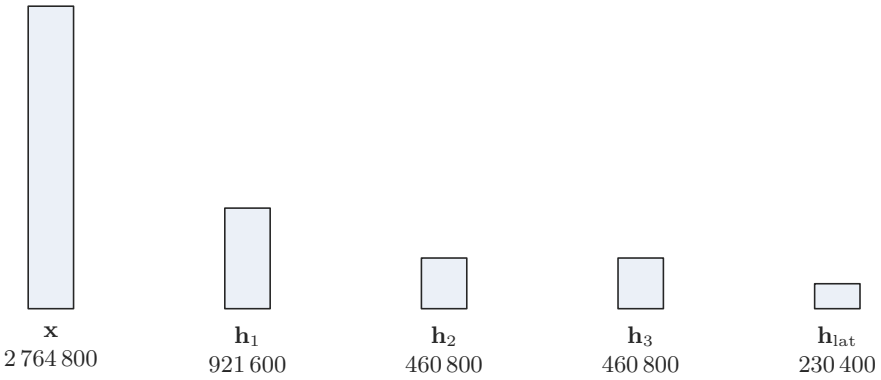
Two teams, A and B, each design a neural network for a challenging task: compressing a 720p HD color image (720×1280 pixels). The image is represented using three channels corresponding to the red, green, and blue components, $\mathbf{x} \in \mathbb{R}^{3\times720\times1280}$.

Both teams use an encoder network and a decoder network. The first network, the *encoder*, transforms the input image into a low-dimensional latent representation $\mathbf{h}_{\text{lat}}$, which constitutes the compressed representation. The second network, the *decoder*, reconstructs the original image as faithfully as possible from this low-dimensional representation. We define the *compression ratio* η of an encoder as the number of values of the image divided by the number of values of its latent representation.

Team A uses a multilayer perceptron with ReLU activations for its encoder. It flattens the image into a single vector before feeding it to the network, and each block k of its encoder consists of a linear projection followed by the activation:

$$\mathbf{h}_k = \sigma\left(\mathbf{W}_k \mathbf{h}_{k-1} + \mathbf{b}_k\right), \quad \mathbf{h}_0 = \mathbf{x}.$$

The dimensions of the different layers of team A's encoder are given below:



(a) How many trainable parameters does team A's encoder contain? Is this network realistic to implement on a standard laptop? Assume that each parameter is encoded using 32 bits.

Solution. The weights and biases of the four layers have shapes

$$\begin{aligned}
\mathbf{W}_1 &\in \mathbb{R}^{921\,600\times2\,764\,800}, & \mathbf{b}_1 &\in \mathbb{R}^{921\,600},\\
\mathbf{W}_2 &\in \mathbb{R}^{460\,800\times921\,600}, & \mathbf{b}_2 &\in \mathbb{R}^{460\,800},\\
\mathbf{W}_3 &\in \mathbb{R}^{460\,800\times460\,800}, & \mathbf{b}_3 &\in \mathbb{R}^{460\,800},\\
\mathbf{W}_4 &\in \mathbb{R}^{230\,400\times460\,800}, & \mathbf{b}_4 &\in \mathbb{R}^{230\,400}.
\end{aligned}$$

The total number of parameters is:

$$\begin{aligned}
N_{\text{param}} &= 921600 \times 2764800 + 921600 + 460800 \times 921600 + 460800\\
&\quad + 460800 \times 460800 + 460800 + 230400 \times 460800 + 230400\\
&= \boxed{3\,291\,219\,993\,600}.
\end{aligned}$$

Thus, using 32 bits (4 bytes) per parameter, the memory required to store the parameters is

$$M = 3\,291\,219\,993\,600 \times 4 \text{ bytes} = \frac{13\,164\,879\,974\,400}{2^{40}} \text{ TiB}$$
$$\approx \boxed{11.97 \text{ TiB}}.$$

Clearly, team A's encoder is infeasible.

(b) What is the compression ratio achieved by team A's encoder?

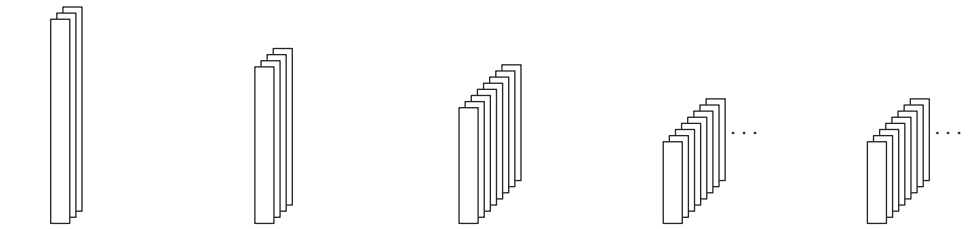
Solution.

$$\eta = \frac{3 \times 720 \times 1280}{230\,400} = \frac{2\,764\,800}{230\,400} = 12 :$$

the latent representation holds 1/12 of the values of the image, a saving of $1 - 1/12 \approx 92\%$. This counts values, not bits: if the pixels are stored on 8 bits and the latent values as 32-bit floats, the ratio in bits is only $12 \times 8/32 = 3$.

Team B uses a convolutional neural network, also with ReLU activations, for its encoder.

Each block of the network consists of a convolutional layer with 3×3 kernels, followed by a ReLU activation and, in every block but the last, a max-pooling layer with a pooling area of 2×2. Before being passed through each block, the input feature maps of each block are padded with a one-pixel-wide border of zeros. The shapes of the different layers of team B's encoder are given below, as channels × height × width:



(c) How many parameters does team B's encoder contain? Is this network realistic to implement on a standard laptop? Assume that each parameter is encoded using 32 bits.

Solution. Let the input of a block be

$$\mathbf{h} \in \mathbb{R}^{a \times b \times c},$$

where a denotes the number of input channels, and b and c denote the height and width of the feature maps, respectively.

Before the convolution, the input is padded with a border of zeros of width 1. The padded input therefore has shape

$$\mathbf{h}_p \in \mathbb{R}^{a \times (b+2) \times (c+2)}.$$

A convolutional kernel of spatial size 3×3 must span all a input channels. Consequently, each kernel has shape

$$a \times 3 \times 3$$

and therefore contains

$$9a$$

trainable weights. If the convolutional layer contains k kernels, the total number of weights is

$$\boxed{9ak}.$$

As each kernel also has a bias, the total number of trainable parameters is

$$\boxed{9ak + k = k(9a + 1)}.$$

Because the input is padded by one pixel on each side and the convolution uses a 3×3 kernel, the spatial dimensions are preserved:

$$(b + 2) - 3 + 1 = b, \qquad (c + 2) - 3 + 1 = c.$$

Therefore, the output of the convolutional layer has shape

$$k \times b \times c.$$

The ReLU does not modify the dimensions. Finally, the 2×2 max-pooling layer reduces each spatial dimension by a factor of 2. Assuming b and c are even, the output of the block has shape

$$\boxed{k \times \frac{b}{2} \times \frac{c}{2}},$$

except for the last block, which has no pooling and keeps the shape $k \times b \times c$. The number of kernels and their shapes throughout the network are:

1. 4 kernels of shape $3 \times 3 \times 3$,
2. 8 kernels of shape $4 \times 3 \times 3$,
3. 32 kernels of shape $8 \times 3 \times 3$,
4. 16 kernels of shape $32 \times 3 \times 3$.

The total number of parameters is therefore

$$\begin{aligned} N_{\text{param}} &= 4(3 \times 3 \times 3 + 1) + 8(4 \times 3 \times 3 + 1) \\ &\quad + 32(8 \times 3 \times 3 + 1) + 16(32 \times 3 \times 3 + 1) \\ &= 4(28) + 8(37) + 32(73) + 16(289) \\ &= 112 + 296 + 2336 + 4624 \\ &= \boxed{7368}. \end{aligned}$$

Thus, using 32 bits (4 bytes) per parameter, the memory required to store the parameters is

$$\begin{aligned} M &= 7368 \times 4 \text{ bytes} = 29\,472 \text{ bytes} \\ &= \frac{29\,472}{2^{10}} \text{ KiB} \\ &\approx \boxed{28.8 \text{ KiB}}. \end{aligned}$$

Team B's encoder clearly fits in a laptop.

(d) What is the compression ratio achieved by team B's encoder?

Solution.

$$\eta = \frac{3 \times 720 \times 1280}{16 \times 90 \times 160} = \frac{2\,764\,800}{230\,400} = 12,$$

the same as team A's, with about 4×10^8 times fewer parameters.

(e) On how many pixels of the image does one value of the latent representation depend, for each team?

Solution. For team A, every value of $\mathbf{h}_{\text{lat}}$ depends on every pixel of the image, since each linear layer connects all its inputs to all its outputs.

For team B, a value depends only on its *receptive field*, a square patch of the image (in all three channels). We follow its size r through the network, together with the distance s in pixels between two neighbouring values of the current layer: a 3×3 convolution adds $2s$ to r, and a 2×2 max-pooling adds s to r and doubles s. Starting from $r = s = 1$:

	conv.	pool.	conv.	pool.	conv.	pool.	conv.
r	3	4	8	10	18	22	38
s	1	2	2	4	4	8	8

One value of team B's latent representation depends on a 38×38 patch of the image (less at its borders, where the patch includes padding), $3 \times 38 \times 38 = 4332$ of its 2 764 800 values. Each value describes a local region of the image, which is why the encoder needs so few parameters.

Exercise 13 Learning to play Pacman (Exam, August 2020)
You observe a Grandmaster agent playing Pacman. How can you use the moves you observe to train your own agent?

(a) Describe formally the data you would collect, the inference problem you would consider, and how you would solve it.

Solution. At each step t of the games we watch, the Grandmaster perceives e_t and plays $a_t \in \mathcal{A} = \{\text{North}, \text{South}, \text{East}, \text{West}, \text{Stop}\}$; Pacman's world is fully observable, so $e_t = s_t$. Each step gives an input-output pair $(\mathbf{x}_i, y_i)$, with features $\mathbf{x}_i = g(e_t) \in \mathbb{R}^d$ computed from the game state (positions of Pacman, the ghosts and the food, distances, walls around Pacman, ...) and the label $y_i = a_t$. Collecting N steps gives a dataset $\mathbf{d} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$. The inference problem is a classification problem with $|\mathcal{A}|$ classes: we look for a model $P_\theta(y \mid \mathbf{x})$ of the Grandmaster's choice given the features, whose parameters θ maximize the likelihood of $\mathbf{d}$,

$$\theta^* = \arg\max_\theta \prod_{i=1}^{N} P_\theta(y_i \mid \mathbf{x}_i).$$

Since there is no closed-form solution for a neural network, we minimize the negative log-likelihood by (stochastic) gradient descent. Our agent is then the simple reflex agent $\pi_\theta(e_t) = \arg\max_{a \in \mathcal{A}} P_{\theta^*}(a \mid g(e_t))$, whose condition-action rules are learned from the Grandmaster rather than written by hand.

(b) How would you design a neural network to control your agent? Define mathematically the neural network architecture, its inputs, its outputs, its parameters, as well as the loss you would use to train it.

Solution. A multilayer perceptron with input $\mathbf{x} = g(e_t) \in \mathbb{R}^d$, K hidden layers and $C = |\mathcal{A}| = 5$ outputs:

$$\begin{aligned} \mathbf{h}_0 &= \mathbf{x}, \\ \mathbf{h}_k &= \text{ReLU}(\mathbf{W}_k \mathbf{h}_{k-1} + \mathbf{b}_k), \quad k = 1, \dots, K, \\ f(\mathbf{x}; \theta) &= \text{Softmax}(\mathbf{W}_y \mathbf{h}_K + \mathbf{b}_y), \end{aligned}$$

with $\mathbf{W}_k \in \mathbb{R}^{q_k \times q_{k-1}}$, $\mathbf{b}_k \in \mathbb{R}^{q_k}$ ($q_0 = d$), $\mathbf{W}_y \in \mathbb{R}^{C \times q_K}$ and $\mathbf{b}_y \in \mathbb{R}^C$, so that $\theta = \{\mathbf{W}_1, \mathbf{b}_1, \dots, \mathbf{W}_K, \mathbf{b}_K, \mathbf{W}_y, \mathbf{b}_y\}$. The output $f_a(\mathbf{x}; \theta)$ is the probability $P_\theta(a \mid \mathbf{x})$ that the

Grandmaster plays a. Numbering the actions $0, \ldots, C-1$, the loss is the cross-entropy, the negative log-likelihood of $\mathbf{d}$:

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log f_{y_i}(\mathbf{x}_i; \theta).$$

Illegal actions (into a wall) can be masked by setting their probability to zero before taking the arg max.

(c) Discuss the expected performance of the resulting agent when 1. the Grandmaster agent is optimal, and 2. the Grandmaster agent is suboptimal.

Solution. 1. If the Grandmaster is optimal, our agent is at best as good as the Grandmaster, and in practice worse. It only imitates the Grandmaster on the states the Grandmaster visits: once a small error leads it to a state that never appears in $\mathbf{d}$, its predictions there are unreliable and errors can compound over the rest of the game. More data, from varied situations, and features g that generalize across mazes reduce this effect. 2. If the Grandmaster is suboptimal, our agent learns its mistakes along with its good moves, since the loss only rewards agreeing with the Grandmaster. Nothing in the data tells which moves were good: doing better requires a signal about the outcome of the moves, such as the score, which is the setting of reinforcement learning.

Quiz

1. We build a MLP with 2 hidden layers of 32 units, ReLU activations and squared error loss. We discard the biases from our linear layers. We know that our input is a vector of 2 elements and the output is a scalar. How many parameters do we have in the network?
   - ○ 1024.
   - ● 1120.
   - ○ 1185.
   - ○ Another value than the ones proposed above.
2. We add biases in the network proposed above. What is the number of parameters of the network?
   - ○ Another value than the ones proposed below.
   - ○ 1089.
   - ● 1185.
   - ○ 2048.
   - ○ 2240.
3. We have an image of shape (H, W), we want to pass it through a layer which reduces each dimension by a constant K, *i.e.* produces an output image of shape $(H-K, W-K)$.
   - ○ It can be done with a convolution layer with a kernel (K, K) or a linear layer with weight matrix $(HW, (H-K)(W-K))$.
   - ● It can be done with a convolution layer with a kernel $(K+1, K+1)$ or a linear layer with weight matrix $(HW, (H-K)(W-K))$.
   - ○ It can be done with a convolution layer with a kernel $(K-1, K-1)$ or a linear layer with weight matrix (HW, KK).
   - ○ It can be done with a convolution layer with a kernel $(K+1, K+1)$ or a linear layer with weight matrix (HW, KK).

4. From the previous question, we double the number of input pixels by setting the shape of the input image to (H', W) with $H' = 2H$. The latter implies that the output image should be of shape $(H' - K, W - K)$. If we only look at weight parameters (no biases)...
   - ◯ The number of parameters of both the linear and convolution layer are doubled.
   - ◯ The linear layer will have twice more parameters and the convolution twice less.
   - ◉ The linear layer will have at least four times more parameters and the convolution will not change.
   - ◯ Nothing changes neither for the linear layer nor the convolution one.
5. In deep learning, a layer in a multi-layer perceptron is defined as ...
   - ◯ $\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$, where σ is the standard deviation function.
   - ◉ $\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$, where σ is an activation function, such as the sigmoid function.
   - ◯ $\mathbf{h} = \sigma(\mathbf{W} + \mathbf{x} - \mathbf{b})$, where $\mathbf{W} \in \mathbb{R}^{q \times d}$ is matrix of weights.
   - ◯ $\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$, where $\mathbf{b} \in \mathbb{R}^d$ is the most likely vector of hidden states given $\mathbf{x}$.
6. Arnaud is trying to perform gradient descent on a function $f(x)$ using the update

$$x_{t+1} := x_t - \frac{\partial f}{\partial x}(x_t).$$

   Is gradient descent guaranteed to converge to the global minimum of f?
   - ◯ Yes, since he's updating using the gradient of x.
   - ◯ Yes, but not for the reason above.
   - ◯ No, since he is updating x in the wrong direction.
   - ◉ No, but not for the reason above.